Enhancing Distance-Based Graph Autoencoders with Structural Penalties for Dynamic Graph Embedding

Aleksandar Tomčić, Miloš Savić, and Miloš Radovanović

Department of Mathematics and Informatics, Faculty of Sciences,
University of Novi Sad, Serbia
Trg Dositeja Obradovića 4, 21000 Novi Sad, Serbia
{atomic, svc, radacha}@dmi.uns.ac.rs

Abstract. Graph autoencoders (GAEs) are widely used for learning representations of dynamic graphs. However, their optimisation objectives typically do not take structural heterogeneity across nodes into account. We propose three distance-based GAE variants that incorporate structural penalties into the reconstruction loss. All variants share a two-layer Graph Convolutional Network encoder and a Euclidean-distance decoder trained with distance-based reconstruction objectives. We extend sparsity-corrected loss with two node-level regularization terms: (i) a hub penalty based on degree centrality, and (ii) a penalty based on Natural Community Local Intrinsic Dimensionality (NC-LID). The paper is motivated by prior evidence linking high NC-LID to reduced embedding quality. The proposed methods are designed to emphasize reconstruction errors for structurally ambiguous nodes. Experiments on multiple dynamic graph data sets show that incorporating NC-LID-based regularization consistently improves reconstruction performance over the baseline without structural regularization and the method using hub-aware regularization. These findings highlight NC-LID as a useful structural signal for enhancing distance-based graph autoencoders in dynamic settings.

Keywords: Dynamic graph embedding, Graph autoencoder, Hubness, Local intrinsic dimensionality (LID), Euclidean distance decoder, Structural regularization

1 Introduction

Many real-world complex systems are inherently dynamic. The networks that represent them evolve over time through the addition or removal of nodes and edges [3]. Learning useful representations of such evolving structures is a core challenge in machine learning on graphs [1]. Graph embedding methods offer a task-agnostic solution, as they map nodes to low dimensional vectors that preserve structural properties, and can subsequently be used for a wide range of downstream tasks including link prediction, node classification, and anomaly detection [4].

Among dynamic graph embedding approaches, two main families exist. Methods based on random walks [9, 10, 12] generate node contexts by traversing the graph and learn embeddings from the resulting sequences, following the paradigm of static walk-based methods such as node2vec [2] and DeepWalk [13]. Methods based on autoencoders [1] encode the graph structure directly through a neural network and reconstruct it from a low-dimensional bottleneck. While walk-based methods have received more research attention, autoencoder-based methods remain comparatively underexplored in terms of structural signals that could improve their training objectives.

Recent lines of work are particularly relevant here. First, it has been shown that hub-awareness, incorporating node degree centrality into walk transition probabilities, substantially improves dynamic graph embedding [18]. Second, NC-LID, a graph-adapted measure of local intrinsic dimensionality [15, 16], has been shown to negatively correlate with node embedding quality in dynamic graphs. Nodes with high NC-LID (complex, irregularly-shaped natural communities) tend to be systematically underrepresented in walk-based dynamic embeddings [7]. Both findings point to structural properties that the standard training objectives of autoencoder-based methods do not take into account.

This paper investigates whether these structural signals can be incorporated directly into the loss function of a distance-based graph autoencoder. We make the following contributions:

- We design a **Euclidean distance decoder** for dynamic graph autoencoders that directly aligns the training objective with the Euclidean distance-based graph reconstruction criterion used in evaluation, correcting the geometric mismatch present in the standard inner product decoder.
- We introduce two **structurally penalised loss functions**, one based on the degree outer product (hub penalty) and one based on the NC-LID outer product (LID penalty), each amplifying reconstruction errors for structurally significant node pairs.
- We provide a systematic empirical evaluation on nine dynamic networks, showing that the LID-penalised variant achieves better reconstruction F_1 score in six of nine datasets while adding negligible computational overhead through a precomputation strategy for NC-LID scores.

2 Related Work

Surveys by Barros et al. [1] and Khoshraftar and An [4] provide comprehensive overviews of dynamic graph embedding methods. Among walk-based methods, Dynnode2vec [9] initialises embeddings via node2vec [2] and incrementally updates only nodes whose ego networks change; CTDNE [10] enforces temporal ordering within walks, and STWalk [12] combines spatial and historical walks. Autoencoder-based methods are surveyed in [1], but remain comparatively underexplored in terms of structural inductive biases.

In the static setting, hub-aware random walk methods [17] that adjust transition probabilities by node degree improve node classification performance; DeepHub [18] extends this to the dynamic setting, showing that inverse-degree-biased walks outperform Dynnode2vec on graph reconstruction while narrowing the hub/non-hub embedding quality gap.

Recent work [15, 16] introduced NC-LID, a local intrinsic dimensionality measure defined over natural communities [8]: low NC-LID indicates compact, well-defined communities, while high NC-LID indicates structurally ambiguous nodes spanning multiple structural directions. LID-elastic node2vec [16] improved both intrinsic embedding quality and downstream performance in the static setting, and [7] adapted NC-LID to the dynamic setting, showing significant negative Spearman correlations between NC-LID and embedding F_1 under Dynnode2vec, establishing NC-LID as a reliable indicator of weakly embedded nodes.

Kipf and Welling [6] introduced the GAE framework, combining GCN encoders [5] with inner product decoders. This decoder has become standard, but its implicit assumption, that co-directional vectors correspond to connected nodes, is inconsistent with Euclidean distance-based evaluation. Distance-based decoders have been explored for hyperbolic embeddings [11] and link prediction [19], but not, to our knowledge, systematically applied to dynamic graph autoencoders with structurally informed losses.

3 Proposed Methods

3.1 Background: Graph Autoencoders and GCNs

An autoencoder is a neural network trained to reconstruct its own input through a low-dimensional bottleneck. The encoder $f_\theta : \mathcal{X} \to \mathcal{Z}$ maps an input $x \in \mathcal{X}$ to a latent code $z \in \mathcal{Z}$, and the decoder $g_\phi : \mathcal{Z} \to \mathcal{X}$ attempts to recover x from z:

$$\min_{\theta,\phi} \; \mathbb{E}_{x\sim p(x)}[\mathcal{L}(x,\, g_\phi(f_\theta(x)))]\,. \tag{1}$$

Since $|\mathcal{Z}| \ll |\mathcal{X}|$, the model must discard noise and retain only the most salient structure of the input.

A Graph Autoencoder (GAE) [6] instantiates this with a GCN encoder [5]. At layer ℓ, node representations are updated by:

$$\mathbf{H}^{(\ell)} = \sigma\Big(\tilde{\mathbf{A}}\,\mathbf{H}^{(\ell-1)}\,\mathbf{W}^{(\ell)}\Big)\,, \tag{2}$$

where $\tilde{\mathbf{A}} = \hat{\mathbf{D}}^{-1/2}(\mathbf{A}+\mathbf{I})\hat{\mathbf{D}}^{-1/2}$ is the symmetrically normalised adjacency with self-loops and $\mathbf{W}^{(\ell)}$ is a learnable weight matrix. After L layers, the embedding of node v_i integrates the structure of all nodes within graph distance L, e.g., two layers encode both immediate and two-hop neighbourhood structure.

3.2 Dynamic Graph Setting

Let $\mathcal{G} = \{G_1, \ldots, G_T\}$ denote a discrete-time dynamic graph, where each snapshot $G_t = (V_t, E_t)$ is a static graph on $n_t = |V_t|$ nodes, and let $\mathcal{V} = \bigcup_t V_t$ be the global vocabulary of $N = |\mathcal{V}|$ distinct nodes. Since these datasets carry no semantic node attributes, each node is represented by a one-hot identity vector keyed by its global vocabulary index, giving feature matrix $\mathbf{X}^{(t)} \in \{0,1\}^{n_t \times N}$ with $X^{(t)}_{i,\mathtt{idx}(v_i)} = 1$ elsewhere zero. This global vocabulary keeps the encoder input dimension consistent across snapshots without padding or masking, regardless of node dynamics.

3.3 Euclidean Distance Decoder

Classical GAEs decode via the inner product $\hat{A}_{ij} = \sigma(\mathbf{z}_i^\top \mathbf{z}_j)$, assigning high scores to co-directional rather than geometrically close vectors. Since graph reconstruction evaluation connects the $|E_t|$ closest node pairs by Euclidean distance [18, 7], training with an inner product decoder mismatches the objective and the evaluation metric.

Our **EuclideanGAEModel** replaces the inner product decoder with one based on pairwise squared Euclidean distances. The encoder uses tanh in the hidden layer to bound embedding coordinates and prevent distance explosion:

$$\mathbf{H} = \tanh\Big(\tilde{\mathbf{A}}\,\mathbf{X}\,\mathbf{W}^{(1)}\Big), \tag{3}$$

$$\mathbf{Z} = \tilde{\mathbf{A}}\,\mathbf{H}\,\mathbf{W}^{(2)}, \tag{4}$$

with $\mathbf{W}^{(1)} \in \mathbb{R}^{N \times d_h}$, $\mathbf{W}^{(2)} \in \mathbb{R}^{d_h \times d}$, and hidden dimension $d_h = 2d$. The decoder forms the full pairwise distance matrix using the numerically stable expansion:

$$D_{ij} = \max\big(0,\ s_i + s_j - 2[\mathbf{Z}\mathbf{Z}^\top]_{ij}\big), \quad s_i = \textstyle\sum_k Z_{ik}^2, \tag{5}$$

and converts distances to adjacency logits via a learnable scalar bias b (initialised to zero):

$$\hat{A}_{ij} = b - D_{ij}. \tag{6}$$

Node pairs with $D_{ij} < b$ receive positive logits (edge predicted); pairs with $D_{ij} > b$ receive negative logits (no edge predicted). The bias adapts to the average connectivity of each snapshot. The computational cost of the decoder is $O(n^2 d)$, identical to the inner product decoder, since the dominant operation is $\mathbf{Z}\mathbf{Z}^\top$.

3.4 Sparsity-Corrected Baseline Loss

Real-world networks are highly sparse, the edge density $|E_t|/n_t^2$ is below 1% for most datasets studied here. A naïve binary cross entropy (BCE) loss is dominated by the abundant negative pairs, biasing the model toward predicting no edges.

Following standard practice, we up-weight positive pairs by the class imbalance ratio:

$$w^+ = \frac{n^2 - |E|}{|E| + \varepsilon}, \qquad \varepsilon = 10^{-9}, \tag{7}$$

yielding the sparsity corrected baseline loss used by **Distance_Basic** variant:

$$\mathcal{L}_{\text{base}} = \frac{1}{n^2} \sum_{i,j} w_{ij}\, \ell_{\text{BCE}}\Big(\hat{A}_{ij}, A_{ij}\Big), \quad w_{ij} = \begin{cases} w^+ & A_{ij} = 1 \\ 1 & A_{ij} = 0. \end{cases} \tag{8}$$

where $\ell_{\text{BCE}}(\hat{a}, a) = -a \log \sigma(\hat{a}) - (1 - a) \log(1 - \sigma(\hat{a}))$ and σ is the sigmoid function.

3.5 Hub-Penalty Variant

High-degree nodes (*hubs*) occupy structurally critical positions [14, 17, 18], acting as bridges between communities and appearing disproportionately in shortest paths, so incorrectly placing a hub propagates reconstruction error to all $\deg(v)$ incident pairs. The **Distance_Hub** variant amplifies the reconstruction loss for hub-involved pairs in proportion to the product of their degrees:

$$\delta_{ij} = \deg(v_i) \cdot \deg(v_j), \qquad \tilde{\delta}_{ij} = \frac{\delta_{ij}}{\max_{k,l} \delta_{kl} + \varepsilon}. \tag{9}$$

$$\mathcal{L}_{\text{hub}} = \frac{1}{n^2} \sum_{i,j} w_{ij}\, \ell_{\text{BCE}}\Big(\hat{A}_{ij}, A_{ij}\Big) \cdot \big(1 + \alpha_h\, \tilde{\delta}_{ij}\big), \tag{10}$$

where $\alpha_h \geq 0$ controls penalty strength (set to 1.0 in all experiments). The multiplier $(1 + \alpha_h\, \tilde{\delta}_{ij}) \in [1, 1 + \alpha_h]$ ensures the baseline loss is only amplified, never suppressed.

3.6 LID-Penalty Variant

The **Distance_LID** variant replaces the degree-based signal with NC-LID, a topological measure of structural ambiguity. Following work presented in [16], the NC-LID of a node v is defined over its natural community S (recovered by the fitness-based algorithm [8] with v as seed) and the largest shortest-path distance k from v to any node in S:

$$\text{NC-LID}(v) = -\ln\left(\frac{|S|}{D(v,k)}\right), \tag{11}$$

where $D(v, k)$ is the number of nodes at shortest-path distance at most k from v. A value of zero indicates a perfectly compact community; higher values indicate that the shortest-path distance struggles to separate S from the rest of the graph, reflecting structural ambiguity. As shown in [7], NC-LID correlates negatively with embedding quality in dynamic graphs for the majority of datasets.

Table 1. Experimental datasets. Max nodes and max edges refer to the largest individual snapshot.

Dataset	Res.	Snaps.	Max nodes	Max edges
radoslaw-email	month	9	151	1,675
ia-hospital-ward-proximity-attr	day	5	54	492
ia-contacts_hypertext2009	day	3	102	1,062
ia-enron-employees	3-months	13	140	823
ia-primary-school-proximity-attr	day	2	241	5,923
fb-forum	month	6	815	5,838
email-Eu-core-temporal	month	18	762	4,518
CollegeMsg	month	7	1,371	6,865
fb-messages	month	7	1,394	9,425

Amplifying the reconstruction penalty for boundary nodes (high NC-LID on both endpoints) forces the encoder to resolve their positions precisely rather than collapsing them into the interior of the nearest dominant community. The loss is:

$$\lambda_{ij} = \text{NC-LID}(v_i) \cdot \text{NC-LID}(v_j), \qquad \tilde{\lambda}_{ij} = \frac{\lambda_{ij}}{\max_{k,l} \lambda_{kl} + \varepsilon}, \tag{12}$$

$$\mathcal{L}_{\text{lid}} = \frac{1}{n^2} \sum_{i,j} w_{ij}\, \ell_{\text{BCE}}\Big(\hat{A}_{ij}, A_{ij}\Big) \cdot \underbrace{\big(1 + \alpha_\ell\, \tilde{\lambda}_{ij}\big)}_{\text{LID multiplier}}, \tag{13}$$

with $\alpha_\ell = 1.0$ throughout. The LID multiplier lies in $[1, 1 + \alpha_\ell]$, matching the hub variant's guarantee that the baseline loss is never suppressed.

NC-LID estimation scales as $O(n \cdot k_s)$ per snapshot (k_s: average natural community size), which would be prohibitive if repeated for every model and dimension. We instead estimate NC-LID once per dataset, cache it as tensors, and load the cached values at training time; the per-epoch loss then adds a single $O(n^2)$ outer product. This keeps **Distance_LID**'s per-epoch training cost within $\pm 10\%$ of the baseline across all datasets.

The complete training procedure is summarised in Algorithm 1.

4 Experimental Setup

The proposed methods are evaluated on nine publicly available temporal networks from SNAP[1] and Network Repository,[2] summarised in Table 1 (institutional email, physical proximity, and online social messaging networks; 54–1,394 nodes per snapshot, 2–18 snapshots).

Effective graph embeddings should allow reconstruction of the original graph: computing the Euclidean distance between every pair of embedding vectors and

[1] https://snap.stanford.edu/data/\#temporal
[2] https://networkrepository.com/dynamic.php

Algorithm 1 Dynamic graph embedding with structural loss penalty

Require: Dynamic graph $\mathcal{G} = \{G_1, \ldots, G_T\}$, model variant $\mathcal{M}$, embedding dimension d, epochs T_e, learning rate η
Ensure: Per-snapshot embeddings $\{\mathbf{Z}^{(1)}, \ldots, \mathbf{Z}^{(T)}\}$

Build global vocabulary $\mathcal{V}$; set $N \leftarrow |\mathcal{V}|$
if $\mathcal{M} =$ DISTANCE_LID **then**
  for $t \leftarrow 1$ **to** T **do**
    Estimate NC-LID scores for all nodes in G_t via Eq. (11)
    Cache as `torch.Tensor` at index t
  end for
end if
for $t \leftarrow 1$ **to** T **do**
  if $|V_t| < 2$ **then**
    continue
  end if
  Compute $\tilde{\mathbf{A}}^{(t)}$, $\mathbf{X}^{(t)}$, $\mathbf{A}^{(t)}$
  Initialise EUCLIDEANGAE$(N, 2d, d)$; Adam(η)
  if $\mathcal{M} =$ DISTANCE_LID **then**
    Load cached NC-LID tensor for snapshot t
  end if
  for epoch $\leftarrow 1$ **to** T_e **do**
    Forward: $\hat{\mathbf{A}}^{(t)}, \mathbf{Z}^{(t)} \leftarrow$ EUCLIDEANGAE$(\mathbf{X}^{(t)}, \tilde{\mathbf{A}}^{(t)})$
    $\mathcal{L} \leftarrow$ `compute_loss`$(\hat{\mathbf{A}}^{(t)}, \mathbf{A}^{(t)}, G_t)$ ▷ Eq. (8), (10), or (13)
    Backpropagate; update weights
  end for
  Store $\mathbf{Z}^{(t)}$
end for

connecting the closest $|E|$ pairs, where $|E|$ is the number of edges in the original graph. Let n denote an arbitrary node and C the number of correctly reconstructed links incident to n. We use:

- Precision – C divided by the number of links n has in the reconstructed graph.
- Recall – C divided by the number of links n has in the original graph.
- F_1 score – The harmonic mean of precision and recall.

Higher values of precision, recall, and F_1 indicate fewer link reconstruction errors for node n. At the graph level, precision, recall, and F_1 scores can be obtained by macro-averaging over all nodes. We report mean F_1 averaged across all temporal snapshots of each dataset.

Five embedding dimensions are evaluated: $d \in \{10, 25, 50, 100, 200\}$ with hidden dimension $d_h = 2d$. All models are trained for $T_e = 300$ epochs with Adam at $\eta = 0.01$. Each model dimension combination is trained from random initialisation without parameter sharing across snapshots. Penalty strengths are fixed at $\alpha_h = \alpha_\ell = 1.0$ for all penalised variants.

Table 2. Mean reconstruction F_1 for *radoslaw-email* (top) and *ia-hospital-ward-proximity-attr* (bottom).

d	Basic	Hub	LID
radoslaw-email			
10	0.4597	0.4565	**0.4777**
25	0.5567	0.5408	**0.5633**
50	0.5917	0.5763	**0.6016**
100	0.6099	0.6000	**0.6229***
200	0.6045	0.5933	**0.6124**
ia-hospital-ward-proximity-attr			
10	0.6819	0.6893	**0.6932**
25	0.7729	0.7628	**0.7792**
50	**0.8166**	0.7920	0.7962
100	**0.8155**	0.7968	0.8132
200	0.8207	0.7891	**0.8222***

5 Results and Discussion

5.1 Per-Dataset Reconstruction F_1

Tables 2–4 report mean reconstruction F_1 for all nine datasets across the five embedding dimensions. Bold entries mark the best model for each dimension-dataset pair. The symbol $\star$ marks the single best configuration per dataset.

Table 5 condenses the results to the single best F_1 per model per dataset.

5.2 Discussion

Reconstruction F_1 increases consistently as d grows from 10 to 100 across all models and datasets, then plateaus or declines at $d = 200$, consistent with a capacity-generalisation trade-off: at $d = 10$ the bottleneck is too tight for structurally distinct nodes, while at $d = 200$ the encoder ($N \times 2d$ parameters) begins to memorise snapshot-specific pairs rather than learning generalisable codes. The $d = 100$ optimum holds for seven of nine datasets; the exceptions (fb-forum, email-Eu-core) are the largest, most variable datasets, where extra capacity still helps.

Distance_LID wins on six of nine datasets and records the top score in 26 of 45 dimension-dataset configurations, most visibly on communication and forum networks with heterogeneous community structure: radoslaw-email (+1.3 pp), fb-forum (+1.0 pp), and enron-employees (+0.4 pp) at d=100. This matches the NC-LID analysis of [7]: the datasets where NC-LID correlates most negatively with embedding quality under Dynnode2vec are those where the LID penalty helps most, since pairs with high NC-LID on both endpoints receive the largest loss amplification, preventing boundary nodes from collapsing into the interior of the nearest dominant community.

Table 3. Mean reconstruction F_1 for *ia-contacts_hypertext2009* (top), *ia-enron-employees* (middle), and *ia-primary-school-proximity-attr* (bottom).

d	Basic	Hub	LID
ia-contacts_hypertext2009			
10	**0.5273**	0.5120	0.5179
25	0.6373	0.6335	**0.6417**
50	**0.6892**	0.6727	0.6873
100	**0.7290***	0.7197	0.7214
200	**0.7136**	0.7026	0.7130
ia-enron-employees			
10	**0.7359**	0.7247	0.7238
25	0.7942	0.7672	**0.7972**
50	0.8033	0.7938	**0.8083**
100	0.8098	0.7999	**0.8140***
200	**0.8084**	0.7945	0.8069
ia-primary-school-proximity-attr			
10	0.7571	**0.7588**	0.7490
25	0.7644	**0.7651**	0.7635
50	**0.7684**	0.7669	0.7636
100	0.7694	0.7686	**0.7706**
200	**0.7718***	0.7706	0.7698

On homogeneous proximity networks (primary-school, hospital), NC-LID scores are uniformly low, so the LID multiplier approaches 1 everywhere and the LID loss nears the baseline, explaining the near-parity there and confirming the signal is most valuable under structural ambiguity.

Distance_Hub wins zero of nine datasets and only 8 of 45 cells. The failure mode is the long-tailed degree-outer-product distribution on scale-free networks: a handful of extreme hub pairs dominate the gradient, sacrificing non-hub representations and distorting the embedding geometry, echoing [18]'s finding that hub-dominant walks hurt non-hub quality. NC-LID avoids this because its scores are bounded and roughly unimodal. Replacing δ_{ij} with $\log(1+\delta_{ij})$ would dampen the tails while preserving rank order, as in DeepHub's logarithmic scaling [18].

CollegeMsg and fb-messages, whose snapshots vary by up to a factor of 12 in node count, show a sharp F_1 drop at $d = 200$ not seen elsewhere, plausibly because the one-hot feature matrix becomes extremely sparse for small snapshots when N is large, poorly conditioning $\mathbf{W}^{(1)} \in \mathbb{R}^{N\times 2d}$ when $n_t \ll N$. No variant dominates both datasets at $d = 100$: **Distance_Basic** remains best on CollegeMsg (0.4641, vs. 0.4482 for **Distance_LID**, -1.59 pp), while **Distance_LID** leads on fb-messages (0.4471 vs. 0.4320, $+1.51$ pp; Table 4). This points to structural node features (degree, clustering coefficient) as a promising alternative to one-hot initialisation for highly variable-size graphs.

Table 4. Mean reconstruction F_1 for *fb-forum*, *email-Eu-core-temporal*, *CollegeMsg*, and *fb-messages*.

d	Basic	Hub	LID
fb-forum			
10	0.3754	0.3562	**0.4098**
25	0.5797	0.5708	**0.5925**
50	0.6374	0.6277	**0.6431**
100	0.6522	0.6439	**0.6622**
200	0.6638	0.6600	**0.6681***
email-Eu-core-temporal			
10	**0.3462**	0.3387	0.3436
25	0.4364	0.4336	**0.4415**
50	0.4654	0.4598	**0.4663**
100	0.4773	0.4680	**0.4789**
200	0.4912	0.4767	**0.4914***
CollegeMsg			
10	0.2062	0.2231	**0.2242**
25	0.3476	**0.3785**	0.3736
50	0.4350	0.4261	**0.4375**
100	**0.4641***	0.4238	0.4482
200	**0.3758**	0.3604	0.3613
fb-messages			
10	0.1840	**0.2033**	0.2030
25	**0.3648**	0.3397	0.3377
50	**0.4213**	0.4132	0.4147
100	0.4320	0.4392	**0.4471***
200	0.3594	0.3516	**0.3818**

6 Conclusions

We introduced three distance-based graph autoencoder variants for dynamic graph embedding, sharing a Euclidean distance decoder that aligns the training objective with the evaluation metric. Augmenting the sparsity-corrected BCE loss with an NC-LID-based penalty, which amplifies errors for structurally ambiguous boundary nodes, consistently improves reconstruction F_1 across nine dynamic networks: **Distance_LID** achieves the best F_1 in six of nine datasets and wins 26 of 45 dimension-dataset comparisons, confirming NC-LID, previously shown useful for walk-based methods [7], as an equally valuable signal within the autoencoder loss.

Distance_Hub consistently underperforms due to a gradient-concentration pathology from the long-tailed degree distribution, illustrating that structural penalties must be bounded to avoid distorting the embedding geometry, a requirement NC-LID satisfies naturally.

Table 5. Best mean reconstruction F_1 per dataset and model. $d^\star$ is the optimal dimension. Bold marks the winner per dataset. *Win count* tallies best F_1 victories across all 9 datasets.

	Basic		Hub		LID	
Dataset	F_1	$d^\star$	F_1	$d^\star$	F_1	$d^\star$
radoslaw-email	0.6099	100	0.6000	100	**0.6229**	100
hospital-ward	0.8207	200	0.7968	100	**0.8222**	200
hypertext2009	**0.7290**	100	0.7197	100	0.7214	100
enron-employees	0.8098	100	0.7999	100	**0.8140**	100
primary-school	**0.7718**	200	0.7706	200	0.7706	100
fb-forum	0.6638	200	0.6600	200	**0.6681**	200
email-Eu-core	0.4912	200	0.4767	200	**0.4914**	200
CollegeMsg	**0.4641**	100	0.4238	100	0.4482	100
fb-messages	0.4320	100	0.4392	100	**0.4471**	100
Win count	3		0		**6**	

Future work includes (i) extending the LID penalty to walk-based methods, (ii) logarithmic compression of the hub penalty to recover its motivation without the gradient pathology, (iii) replacing one-hot initialisation with structural node features for highly variable-size graphs, (iv) combining LID-aware embeddings with hub-aware hybrid random walkers, and (v) evaluating the proposed variants on downstream tasks such as link prediction.

Acknowledgements

This research is supported by the Science Fund of the Republic of Serbia, #7462, Graphs in Space and Time: Graph Embeddings for Machine Learning in Complex Dynamical Systems – TIGRA.

References

1. Barros, C.D., Mendonça, M.R., Vieira, A.B., Ziviani, A.: A survey on embedding dynamic graphs. ACM Computing Surveys (CSUR) **55**(1), 1–37 (2021)
2. Grover, A., Leskovec, J.: node2vec: Scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 855–864 (2016)
3. Holme, P., Saramäki, J.: Temporal networks. Physics reports **519**(3), 97–125 (2012)
4. Khoshraftar, S., An, A.: A survey on graph representation learning methods. ACM Transactions on Intelligent Systems and Technology **15**(1), 1–55 (2024)
5. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907 (2016)
6. Kipf, T.N., Welling, M.: Variational graph auto-encoders. arXiv preprint arXiv:1611.07308 (2016)

7. Knežević, D., Savić, M., Radovanović, M.: Local intrinsic dimensionality for dynamic graph embeddings. In: International Conference on Complex Networks and Their Applications, pp. 374–385. Springer (2024)
8. Lancichinetti, A., Fortunato, S., Kertész, J.: Detecting the overlapping and hierarchical community structure in complex networks. New journal of physics **11**(3), 033,015 (2009)
9. Mahdavi, S., Khoshraftar, S., An, A.: dynnode2vec: Scalable dynamic network embedding. In: 2018 IEEE international conference on big data (Big Data), pp. 3762–3765. IEEE (2018)
10. Nguyen, G.H., Lee, J.B., Rossi, R.A., Ahmed, N.K., Koh, E., Kim, S.: Continuous-time dynamic network embeddings. In: Companion proceedings of the the web conference 2018, pp. 969–976 (2018)
11. Nickel, M., Kiela, D.: Poincaré embeddings for learning hierarchical representations. Advances in neural information processing systems **30** (2017)
12. Pandhre, S., Mittal, H., Gupta, M., Balasubramanian, V.N.: Stwalk: learning trajectory representations in temporal graphs. In: Proceedings of the ACM India joint international conference on data science and management of data, pp. 210–219 (2018)
13. Perozzi, B., Al-Rfou, R., Skiena, S.: Deepwalk: Online learning of social representations. In: Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 701–710 (2014)
14. Radovanovic, M., Nanopoulos, A., Ivanovic, M.: Hubs in space: Popular nearest neighbors in high-dimensional data. Journal of machine learning research **11**(sept), 2487–2531 (2010)
15. Savić, M., Kurbalija, V., Radovanović, M.: Local intrinsic dimensionality and graphs: towards lid-aware graph embedding algorithms. In: International Conference on Similarity Search and Applications, pp. 159–172. Springer (2021)
16. Savić, M., Kurbalija, V., Radovanović, M.: Local intrinsic dimensionality measures for graphs, with applications to graph embeddings. Information Systems **119**, 102,272 (2023)
17. Tomčić, A., Savić, M., Radovanović, M.: Hub-aware random walk graph embedding methods for classification. Statistical Analysis and Data Mining: The ASA Data Science Journal **17**(2), e11,676 (2024)
18. Tomčić, A., Savić, M., Simić, D., Radovanović, M.: Dynamic graph embedding through hub-aware random walks. In: International Conference on Similarity Search and Applications, pp. 315–329. Springer (2025)
19. Zhang, M., Chen, Y.: Link prediction based on graph neural networks. Advances in neural information processing systems **31** (2018)